

School of Engineering and Computer Science

Mechanical Engineering and Mechatronics
program

Computer programming: 4FTC2029-0105

Project title: Multiplayer Memory Game

Team Members:

Name
Youssef Ahmed Galal
Rawan Haytham Anwar
Ahmed Osama Ibrahim
Abdelrahman Taha Elsayed

Table of contents

- 1- Introduction
- 2- System Features
- 3- Details of the Design
- 4- Implementation
- 5- Results
- 6- Analysis
- 7- Conclusion
- 8- References

1) Introduction

Memory games are widely used for entertainment and cognitive training because they improve concentration, reaction speed and short-term memory. Traditional memory games are usually designed for a single player, but adding multiplayer features creates a more engaging and competitive experience.

The aim of this project was to design and implement a microcontroller-based multiplayer memory game using Arduino hardware components. The system uses LEDs, push buttons, an LCD display, a buzzer and EEPROM memory to create an interactive gaming console.

The game includes three different modes: Solo Mode, Duel Mode and Challenger Mode. In each mode, players must memorize and repeat sequences of illuminated LEDs by pressing the corresponding buttons. As the game progresses, the difficulty increases by speeding up the sequence or increasing its length.

This project demonstrates practical programming skills, hardware integration, structured design and real-time input/output control.

2) System Features and Code Structure

2.1) Code Structure

The code is organized into separate files, where each part controls a specific component such as buttons, LEDs, LCD, buzzer, memory, and game modes. The program's setup and loop in `main.ino`, which sets up the hardware, then runs a loop that continuously reads buttons, updates LEDs and buzzer, and controls the system using a state machine. The state machine manages all stages of the game (menu, gameplay, game over, etc.), while each game mode has its own file.

2.2) System Features

The project contained many features including:

- Three different memory game modes:

Solo Sprint → a one player game mode where a random sequence is generated and the player has to replicate it. The game ends when the player times out or inputs the wrong sequence.

Synchronous Duel → a two player game where a random sequence is also generated, but both players race to input that sequence, this mode handles inputs from both players at the same time. The game ends when both players fail to replicate sequence or time out.

Challenger Mode → a two player game where one player inputs a sequence, and the other has to replicate it and gains a point. If the second player fails to do so, player 1 has the ability to gain a point and prove the sequence is possible by inputting it. If both players fail to repeat sequence, no point gained for this round. The game ends when the player inputting the original sequence times out or the first player to reach 10 points.

All game modes include timeouts for the players, 3 difficulties to be chosen before game mode begins (easy, medium, hard), as well as increasing sequences and decreasing LED flashing times to increase difficulty as rounds go on.

- An LCD display that displays a scroll down menu system and displays score during gameplay at all times without flickering. It can also display messages during gameplay such as correct, and P1's turn.
- A non volatile memory that stores the top 5 scores even after shutting down the system, and the high scores can be wiped using an option in the menu.
- Non blocking code (like millis()) is used so the system can remain responsive to both players inputs at the same time.
- Debouncing logic is also used for buttons to prevent double triggers
- Sleep mode is available which turns off backlight of LCD and all LEDs and buzzer if no button is pressed in menu for 2 minutes to save power.
- State Machine is used to handle the different states in the coding for a more structured design.

3) Details of the Design

3.1 System Overview

The system was designed using a modular programming structure where each hardware component and game feature was separated into individual files. This improves readability, debugging and future upgrades.

3.2 Hardware Components

Component	Quantity	Purpose
Arduino Uno	1	Main microcontroller used to control the entire system
LEDs	6	Display the memory sequence visually to the players and an extra to which light up according to player won

Push Buttons	8	Used by players to input the correct sequence
I2C LCD Display	1	Shows menus, scores, messages, and game status
Passive Buzzer	1	Produces sound feedback for correct, wrong, and game over events
320Ω Resistors	6	Protect components and ensure correct current flow
Switch	1	Turn on/off the system
Battery Cells	2	To power the Arduino instead of cable
Battery Holder	1	To hold batteries in place
Breadboard	1	Used to assemble and test the circuit without soldering
Jumper Wires	Multiple	Connect all components to the Arduino

3.3 Functional Modules

Module	Purpose
LEDs.ino	Controls LED flashing sequences
buttons.ino	Reads player input with debouncing
LCD.ino	Displays menus, scores, and messages
Buzzer.ino	Plays feedback sounds
flashMemory.ino	Saves top scores in EEPROM
utils.ino	Random generation and timing
ModeSolo.ino	Solo gameplay logic
ModeDuel.ino	Two-player duel mode
ModeChallenger.ino	Competitive challenge mode
StateMachine.ino	Controls menu and transitions
Config.h	Global settings and constants
Hardware.h	Pin definitions

3.4 State Machine Design

A state machine was used to organize the overall system behavior. The main states were:

- Main Menu
- Difficulty Selection
- Countdown
- Playing

- Success Pause
- Game over
- Sleep Mode

This ensures smooth transitions and organized program flow.

3.5 System Flowchart, Circuit Design and Prototype Images

The following images show the design process, hardware wiring, internal assembly, and final completed product of the Multiplayer Memory Game

System Flowchart

Multiplayer Memory Game – System Flowchart

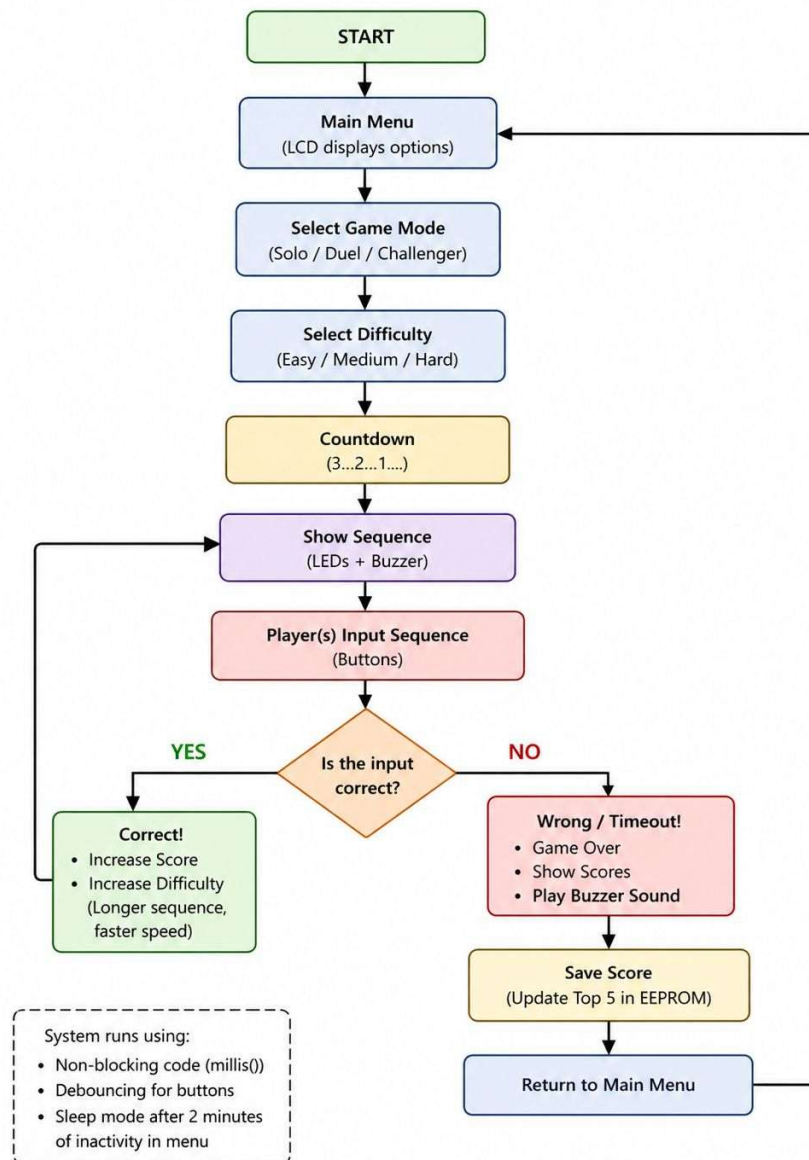


Figure 1: Flowchart of System

This figure displays the flow of the system throughout the loop.

Simulation Circuit Design

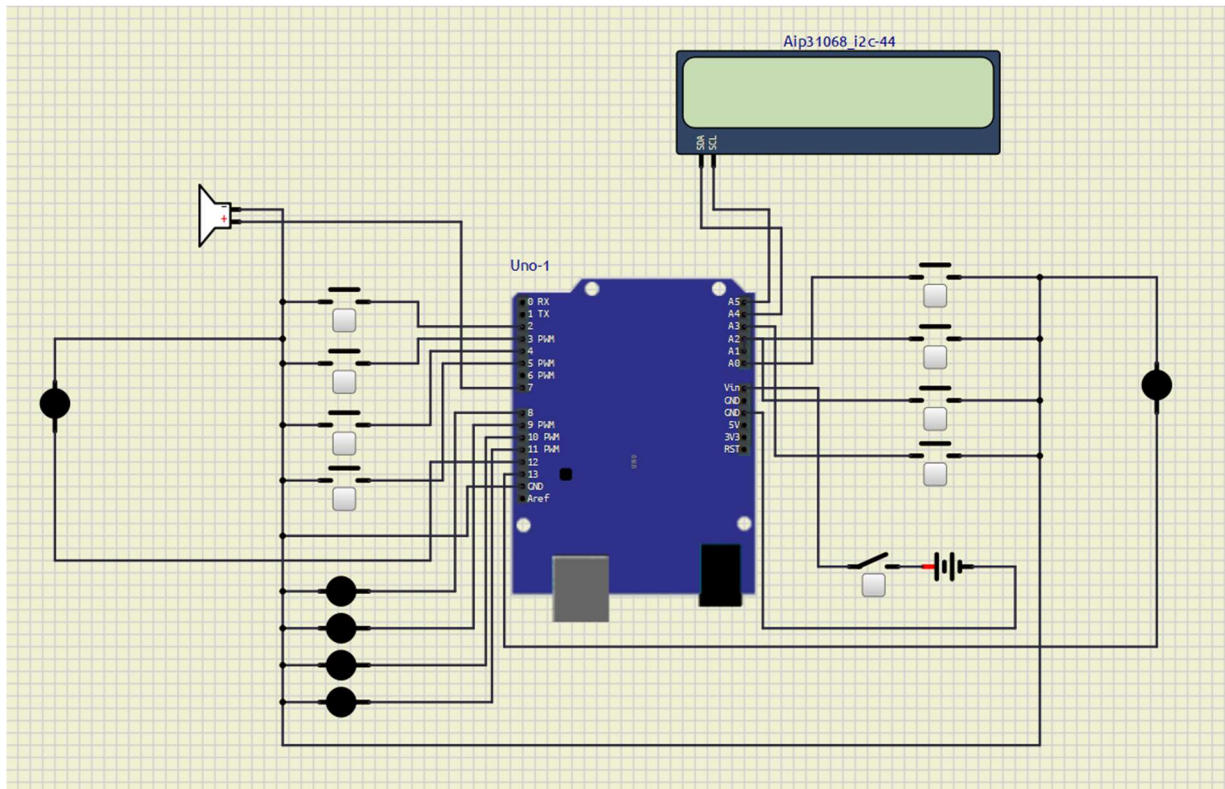


Figure 2: Simulation Circuit Design

This figure shows the planned circuit design in simuIDE including the Arduino uno, LCD display, buttons, LEDs and buzzer connections before physical implementation.

Port D contains player 1 buttons in pins 2,3,4 and 5 as well as buzzer in pin 7.

Port B contains all 6 LEDs with the sequence LEDs in pins 8,9,10 and 11 and two player specific LEDs in pins 12 and 13.

Port C contains player 2 buttons in pins A0,A1,A2 and A3 with pins A4 and A5 connected to the I2C LCD display.

The switch is connected to the battery cells and Vin pin in Arduino.

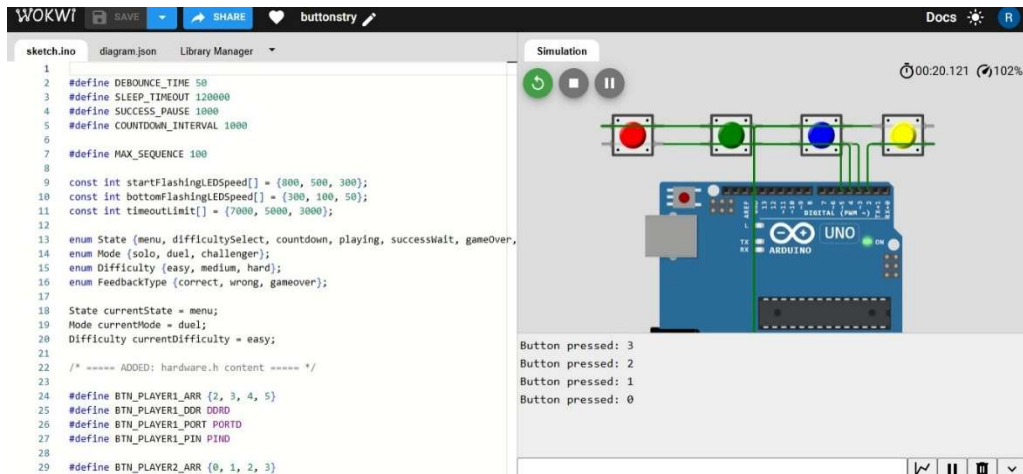


Figure 3: Button Input Testing

This figure shows the test in Wokwi simulation to verify each push button input is detected correctly before connecting to the physical hardware.

Breadboard Prototype Testing

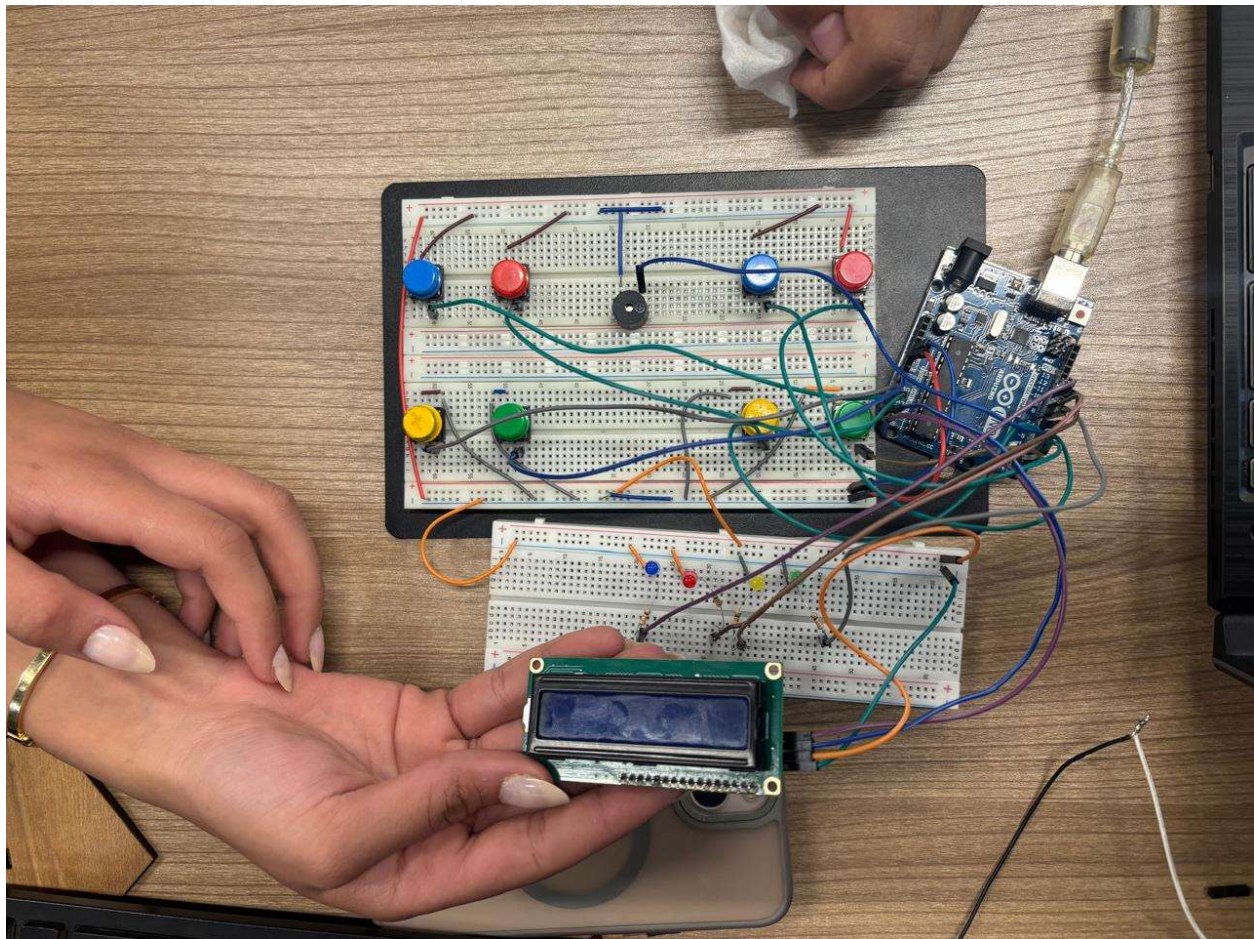


Figure 4: Prototype Breadboard Setup

The first physical prototype was assembled on breadboards to test button input, LED outputs, LCD display functionality and buzzer feedback.

Internal Wiring Layout

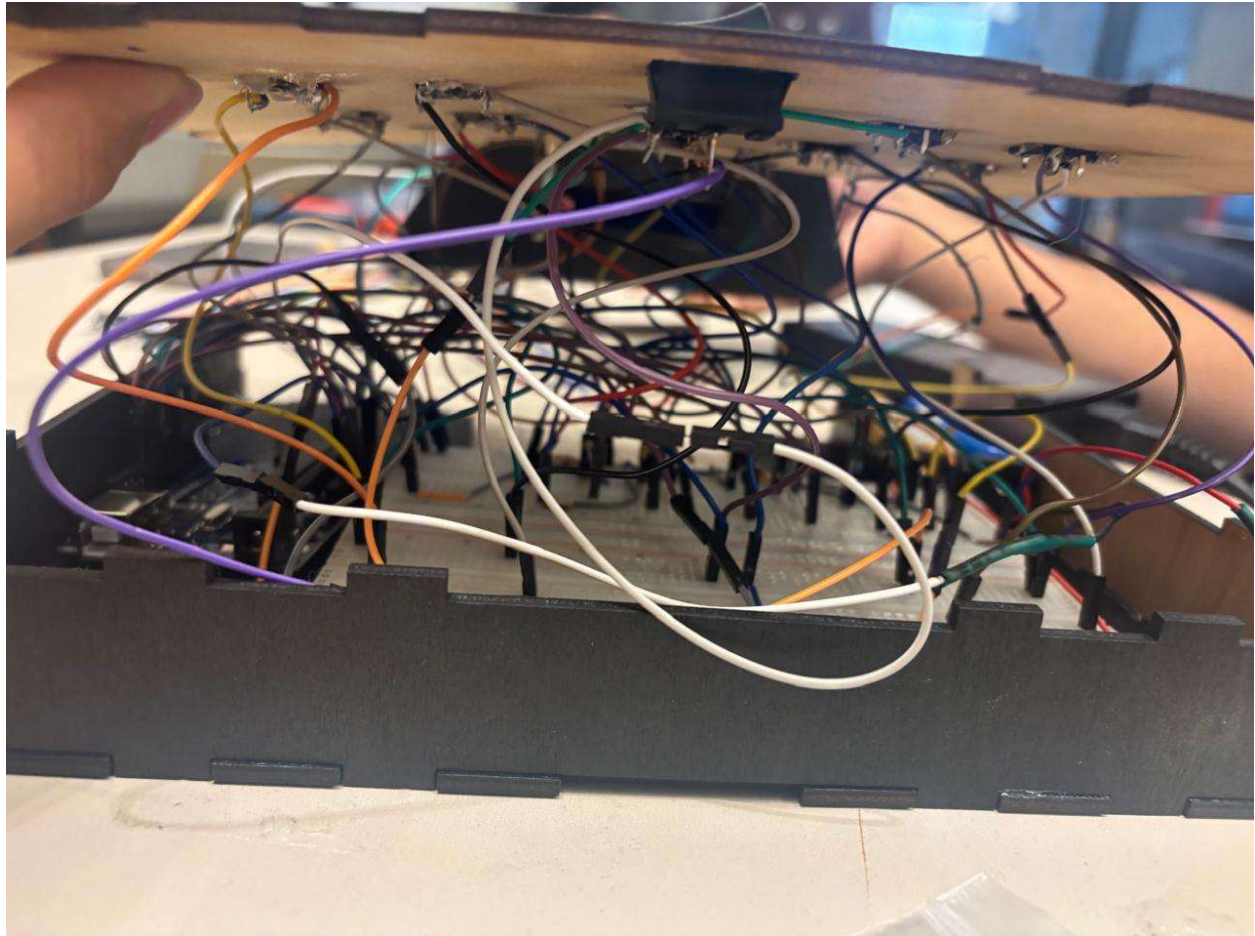


Figure 5: Internal wiring Layout

This image shows the internal wiring and placement of components inside the project box

Figure 4: Final Completed Product



Figure 6: Final Multiplayer Memory Game Console

The final version includes the LCD display, colored push buttons for both players, LED indicators, buzzer and power switch integrated into a custom enclosure.

4) Implementation

4.1 Programming Environment

The project was implemented using Arduino IDE with C programming language.

4.2 Input Handling

The eight push buttons were configured using internal pull-up resistors. Debouncing logic was implemented to prevent false triggering caused by mechanical noise

4.3 LED Sequence Generation

Random LED patterns were generated and stored in arrays. The player must repeat the sequence correctly to continue.

4.4 Display Interface

- The LCD screen was used to display:
- Main menu
- Selected mode
- Countdown timer
- Scores
- Game over messages
- leaderboard

4.5 Sound Feedback

The buzzer provides different sounds for:

- Correct answer
- Wrong answer
- Game over

4.6 EEPROM Storage

The top five high scores were stored in EEPROM memory, allowing scores to remain saved even when power is disconnected.

4.7 Difficulty Levels

Three difficulty levels were implemented:

Difficulty	Start Speed	Minimum Speed	Timeout
Easy	800 ms	300 ms	7s
Medium	500 ms	100 ms	6s
Hard	300 ms	50 ms	5s

This allows the game to be suitable for different skill levels.

5) Results

After finishing each tab in the codes, a test run was made to ensure all functions worked within that tab, then the tests were done to ensure each game mode works separately, and finally, after finishing and verifying code in ArduinoIDE, a full simulation run was made to ensure all states and functions work well in simulIDE. After testing was complete, the code was uploaded to the Arduino, then connected to the breadboard prototype to ensure all hardware components function properly. After some small changes and implementation of the project box, a complete hardware test was done, and it worked as intended and as shown in the video.

6) Analysis

The project successfully combined hardware and software into an interactive embedded system. Using modular programming made the code easier to organize and test.

The use of non-blocking timing functions such as `millis()` improved responsiveness and allowed simultaneous input detection because the system could monitor buttons while LEDs were flashing.

One of the challenges faced was that due to the large code, there were some minor errors in code when testing that were difficult to track down, and required multiple different simulation runs to detect. For example, in the second final test run in simulIDE, one of the game modes took longer than 2 minutes to complete, and therefore, when the system went into game over state, these 2 minutes were detected as the 2 minutes without a button press required to put the system in sleep mode, which resulted in the LCD displaying shutting down immediately after game over and the next game mode after waking back up, not initializing properly. Another challenge was implementing `millis()` instead of `delay()` as this required multiple extra variables to detect the passage of time instead of just writing `delay`.

6.1 Possible Improvements

Future improvements could include:

- Adding time taken to complete sequence or comparing between two player times in each mode
- Saving high scores in each mode instead of overall as well as save the initials for each score
- Adding more game modes
- Adding more sounds from a sound system instead of just a buzzer

7) Conclusion

In conclusion, this project successfully developed a multiplayer memory game using Arduino and external hardware components. The system combined programming logic, user interaction, memory storage and real-time control into one complete embedded application.

The final product met all project objectives by providing multiple game modes, increasing difficulty, competitive gameplay and persistent score storage. The project also improved practical skills in programming debugging, teamwork and hardware integration.

Overall, the project demonstrates how simple electronic components can be combined to create an enjoyable and functional gaming system

8) References

1. Arduino. (2025). *Arduino Uno Rev3 Documentation*. Available at: <https://www.arduino.cc>
2. Monk, S. (2023). *Programming Arduino: Getting Started with Sketches*. McGraw-Hill.
3. Arduino Reference. (2025). *EEPROM Library*. Available at: <https://www.arduino.cc/reference>
4. LiquidCrystal I2C Library Documentation
5. Stephen M. (2019) Intro to Arduino State Machines (Part 1). Available at: <https://www.youtube.com/watch?v=oLor9--g9CE> (Accessed: 29 April 2026).
6. Seeed Studio (2021) Managing Arduino Memory: Flash, SRAM, EEPROM. Available at: <https://www.seeedstudio.com/blog/2021/04/26/managing-arduino-memory-flash-sram-eprom/> (Accessed: 29 April 2026).